

University of Prishtina “Hasan PRISHTINA”



Faculty of Electrical and Computer Engineering
Master in Computer and Software Engineering, 2025-2026
Course: Data Preparation and Visualization

Week X
Understanding Skewness Data

Dr. Sc. Mërgim H. HOTI

Learning objectives

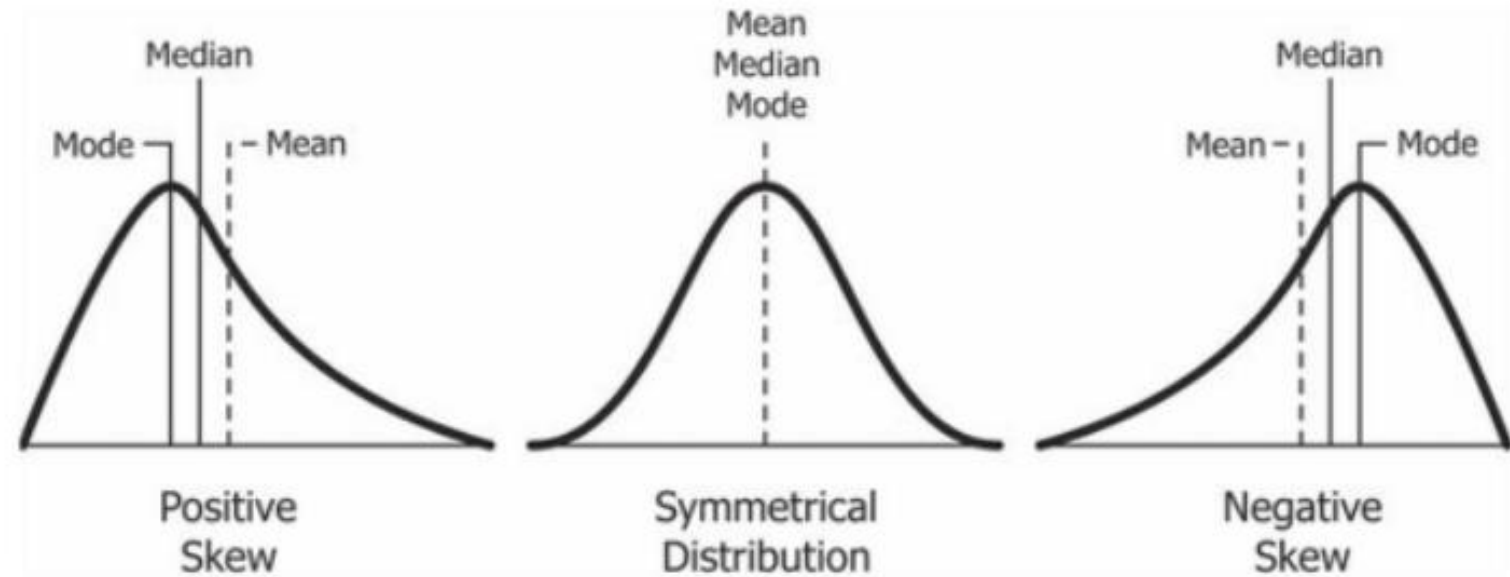
- You'll learn about skewness and its types;
- You will learn what the coefficient of skewness is and how to calculate it;
- Lastly, you will learn how can we transform skewed data;
- Example algorithms to solve this problem “SMOTE/ ADASYN”.

What Is Skewness?

- Skewness is the measure of the asymmetry of an ideally symmetric probability distribution and is given by the third standardized moment;
- If that sounds way too complex, don't worry! Let's we break;
- In simple words, skewness is the measure of how much the probability distribution of a random variable deviates from the normal distribution;
- Now, you might be thinking – why am I talking about normal distribution here?

What Is Skewness? [cont.]

- Well, the normal distribution is the probability distribution without any skewness;
- You can look at the image below, which shows symmetrical distribution that's a normal distribution, and you can see that it is symmetrical on both sides of the dashed line;
- Apart from this, there are two types of skewness:
 - Positive Skewness;
 - Negative Skewness.



What Is Skewness? [cont.]

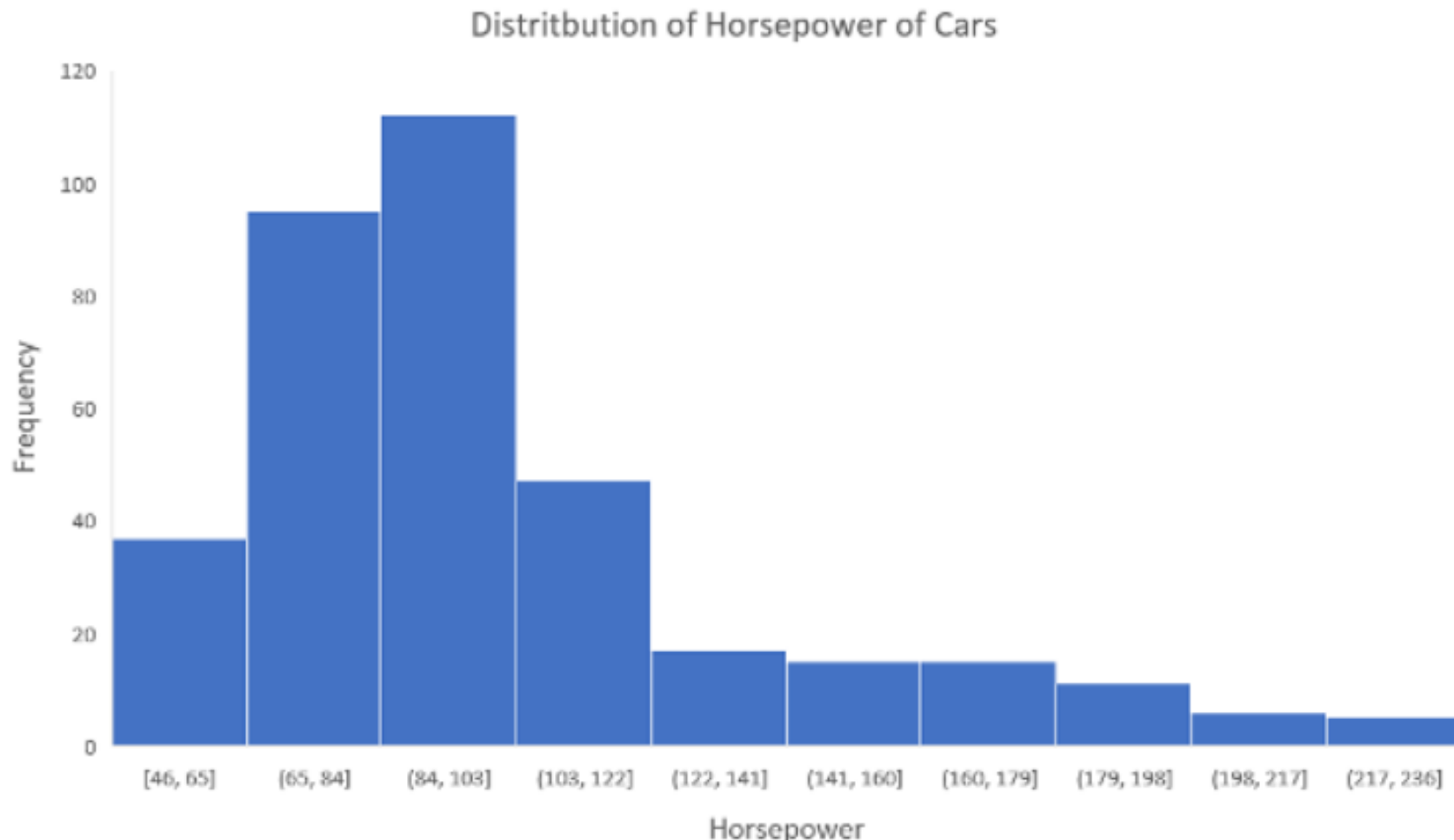
- The probability distribution with its tail on the right side is a positively skewed distribution, and the one with its tail on the left side is a negatively skewed distribution;
- Skewness is demonstrated on a bell curve when data points are not distributed symmetrically to the left and right sides of the median on a bell curve.
- *If the bell curve is shifted to the left or the right, it is said to be skewed;*
- Before that, let's understand why this is such an important concept for our cases.

Why Is Skewness Important?

- Now, we know that skewness is the measure of the lack of symmetry, and its types differ by the side on which the tail of probability distribution lies.
- *But, why is knowing the skewness of the data important?*
- **First**, linear models work on the assumption that the distribution of the **independent variable** and the **target variable** are similar;
- Therefore, knowing about the skewness of data helps us create better linear models.

Why Is Skewness Important?

- **Secondly**, let's take a look at the below distribution. It is the distribution of horsepower of cars:



Why Is Skewness Important? [cont.]

- You can clearly see that the above distribution is positively skewed;
- Now, let's say you want to use this as a feature for the model that will predict the km/l (kilometre per litre) of a car;
- Since our data is **positively** skewed here, it means that **it has a higher number of data points** having low values, i.e., cars with less horsepower.

Why Is Skewness Important? [cont.]

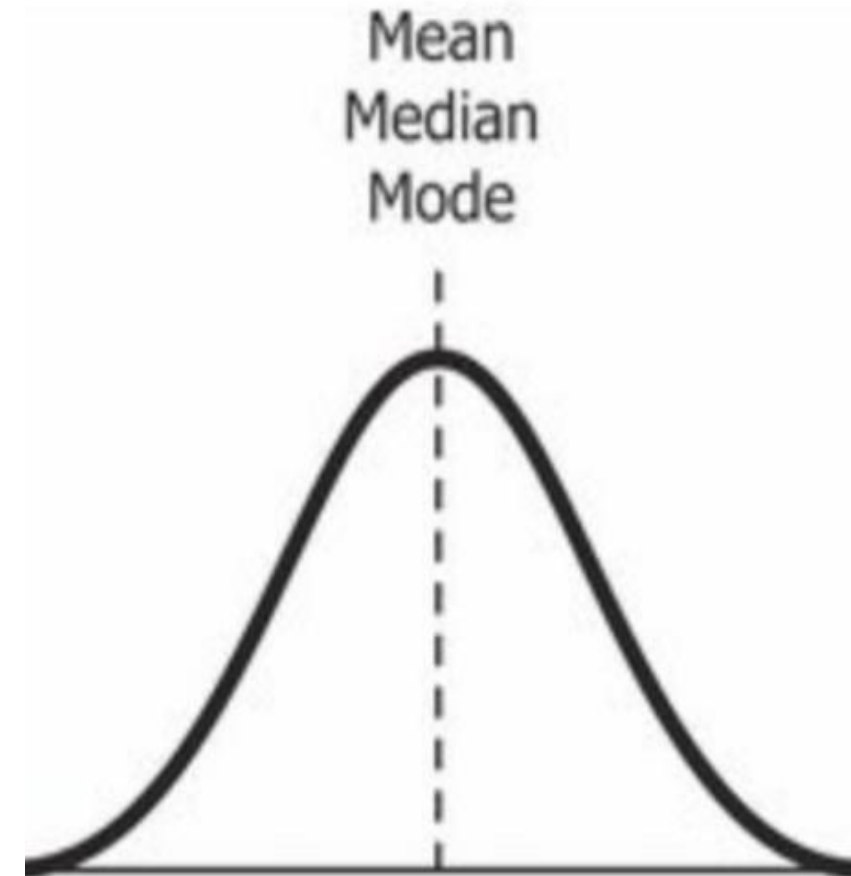
- So, when we train our model on this data, it will perform better at predicting the km/l of cars with lower horsepower as compared to those with higher horsepower;
- Also, skewness tells us about the direction of outliers;
- You can see that our distribution is positively skewed, and most of the outliers are present on the right side of the distribution.

Central Limit Theorem

- The central limit theorem says that the sampling distribution of the mean will always be normally distributed as long as extreme values, or the sample size is large enough;
- Regardless of whether the population has a normal, Poisson, binomial, or any other distribution, the sampling distribution of the mean will be normal;
- Hypothesis Testing uses the Central Limit Theorem (CLT);
- Using the CLT, we can extend the approach employed in Single Sample Hypothesis Testing (SSHT), for normally distributed populations to those that are not normally distributed.

What Is Symmetric/Normal Distribution?

- The normal distribution it's used as a reference for determining the skewness of a distribution;
- As we discussed before, the *ideal normal distribution* is the probability distribution with almost **NO skewness**;
- It is nearly perfectly symmetrical;
- How do we know that the value of skewness is normal?
 - Due to this, the value of skewness for a normal distribution is zero or zero skewness. Do you agree??!!



But why is it nearly perfectly symmetrical and not absolutely symmetrical?

- That's because, in reality, no real word data has a perfectly normal distribution. Therefore, even the value of skewness is **not exactly zero, it is nearly zero**;
- Although the value of zero is used as a reference for determining the skewness of a distribution.
- You can see in the above image that the same line represents the **mean, median, and mode**;
- It is because the **mean, median, and mode** of a perfectly normal distribution are **equal**.
- So far, we've understood the skewness of normal distribution using a probability or frequency distribution.

Common Properties of a Similarity



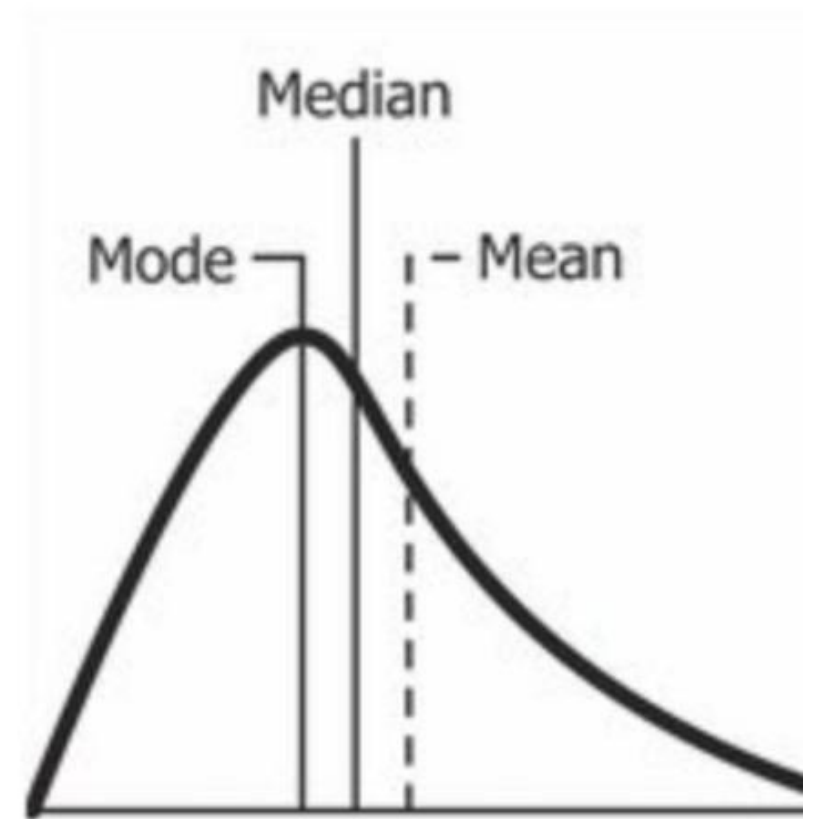
- The above image is a boxplot of symmetric distribution. You'll notice here that the distance between Q1 and Q2 and Q2 and Q3 is equal, i.e.:

$$* \quad Q3 - Q2 = Q2 - Q1$$

- But that's not enough to conclude if a distribution is skewed or not;
- We also take a look at the length of the borders, if they are equal, then we can say that the *distribution is symmetric*, i.e., *it is not skewed*;

Understanding Positively Skewed Distribution | Positive Skewness

- A **positive skewness** is the **right-skewed distribution** with the long tail on its right side;
- The value of skewness for a positively skewed distribution is greater than zero;
- As you might have already understood by looking at the figure, the value of the **mode** is the greatest one, followed by the **median** and then by **mean**.



So why is this happening?

- Also, the mode occurs at the highest frequency of the distribution, which is on the left side of the median;
- Therefore, the measure of central tendencies is:

$$\text{mode} > \text{median} > \text{mean}.$$



- In the above boxplot, you can see that Q2 is present nearer to Q1.
- This represents a positively skewed distribution, in terms of quartiles, it can be given by:

$$Q3 - Q2 > Q2 - Q1$$

So why is this happening? [cont.]



- Here, $Q2-Q1$ and $Q3-Q2$ are **equal**, and yet the distribution is positively skewed;
- Considering previous explanation, you will have noticed the length of the right side is greater than the left side;
- From this, we can conclude that the data is positively skewed:
 - So, the **first step** is always to check the equality of $Q2-Q1$ and $Q3-Q2$;
 - If that is found equal, then we look for the length of the sides.

Understanding Negatively Skewed Distribution | Negative Skewness



- As you might have already guessed, a negatively skewed distribution is the left-skewed distribution with the long tail on its left side;
- The value of skewness for a negatively skewed distribution is less than zero;
- You can also see in the above figure that the measure of central tendencies is:
mean < median < mode



Understanding Negatively Skewed Distribution | Negative Skewness [cont.]



- In the boxplot, the relationship between quartiles for a negative skewness is given by:

$$Q3 - Q2 < Q2 - Q1$$

- Similar to what we did earlier, if $Q3 - Q2$ and $Q2 - Q1$ are equal, then we look for the length of sides;
- And if the length of the left side is greater than that of the right side, then we can say that the data is negatively skewed.



Coefficient of Skewness

➤ Pearson developed two methods to find skewness in a sample.

1. Pearson's Coefficient of Skewness using mode:

$$\mathbf{SK1} = \text{Mean} - \text{mode} / \text{sd} \text{ (standard deviation)}$$

2. Pearson's Coefficient of Skewness using the median:

$$\mathbf{SK2} = 3 (\text{mean} - \text{median}) / \text{sd} \text{ (It is generally used when the mode is unknown)}$$

How Do We Transform Skewed Data?

- Since, we know how much the skewed data can affect our machine learning model's predicting capabilities, it is better to transform the skewed data into normally distributed data;
- Here are some of the ways we can transform skewed data:
 - Power Transformation:
 - *Log Transformation*;
 - *Exponential Transformation*.

SMOTE Algorithm

Problem

- Class imbalance occurs in machine learning when one class of data points is significantly more prevalent than another class;
- This can be a problem when training a machine learning model, as the model may be biased towards the more prevalent class and have difficulty accurately classifying the minority class.

Solutions

- Class imbalance can be addressed using various techniques, such as:
 - ***Oversampling*** (generating new synthetic data points for the minority class) or;
 - ***Under-sampling*** (discarding data points from the majority class).

SMOTE Algorithm

- “SMOTE (Synthetic Minority Oversampling Technique) is a type of oversampling method, which means it works by generating new synthetic data points for the minority class;
- It does this by selecting a minority class data point and finding its k nearest neighbours;
- Then, it generates a new data point by taking the vector between one of these neighbours and the original data point, and multiplying it by a random number between 0 and 1;
- The new data point is then added to the dataset, along with the original data points;
- This process is repeated until the minority class is balanced with the majority class.

Advantages

1. It can significantly improve the performance of a machine learning model on a minority class;
2. It does not require any additional data beyond the original dataset;
3. It is simple to implement and easy to use;
4. It does not discard any of the original data points, unlike under-sampling methods.

Dis-Advantages

1. It can potentially create unrealistic synthetic data points that are not representative of the true distribution of the data;
2. It may not be effective if the minority class is too small or if there are too many dimensions in the data;
3. It can introduce bias into the dataset if the k nearest neighbors are not representative of the minority class as a whole.

SMOTE PseudoCode

Algorithm *SMOTE*(T , N , k)

Input: Number of minority class samples T ; Amount of SMOTE $N\%$; Number of nearest neighbors k

Output: $(N/100) * T$ synthetic minority class samples

```

1.  (* If  $N$  is less than 100%, randomize the minority class samples as only a random
    percent of them will be SMOTEd. *)
2.  if  $N < 100$ 
3.    then Randomize the  $T$  minority class samples
4.         $T = (N/100) * T$ 
5.         $N = 100$ 
6.  endif
7.   $N = (int)(N/100)$  (* The amount of SMOTE is assumed to be in integral multiples of
    100. *)
8.   $k$  = Number of nearest neighbors
9.   $numattrs$  = Number of attributes
10.  $Sample[ ]$ : array for original minority class samples
11.  $newindex$ : keeps a count of number of synthetic samples generated, initialized to 0
12.  $Synthetic[ ]$ : array for synthetic samples
    (* Compute  $k$  nearest neighbors for each minority class sample only. *)
13. for  $i \leftarrow 1$  to  $T$ 
14.     Compute  $k$  nearest neighbors for  $i$ , and save the indices in the  $nnarray$ 
15.      $Populate(N, i, nnarray)$ 
16. endfor

     $Populate(N, i, nnarray)$  (* Function to generate the synthetic samples. *)
17. while  $N \neq 0$ 
18.     Choose a random number between 1 and  $k$ , call it  $nn$ . This step chooses one of
        the  $k$  nearest neighbors of  $i$ .
19.     for  $attr \leftarrow 1$  to  $numattrs$ 
20.         Compute:  $dif = Sample[nnarray[nn]][attr] - Sample[i][attr]$ 
21.         Compute:  $gap$  = random number between 0 and 1
22.          $Synthetic[newindex][attr] = Sample[i][attr] + gap * dif$ 
23.     endfor
24.      $newindex++$ 
25.      $N = N - 1$ 
26. endwhile
27. return (* End of  $Populate$ . *)
    End of Pseudo-Code.

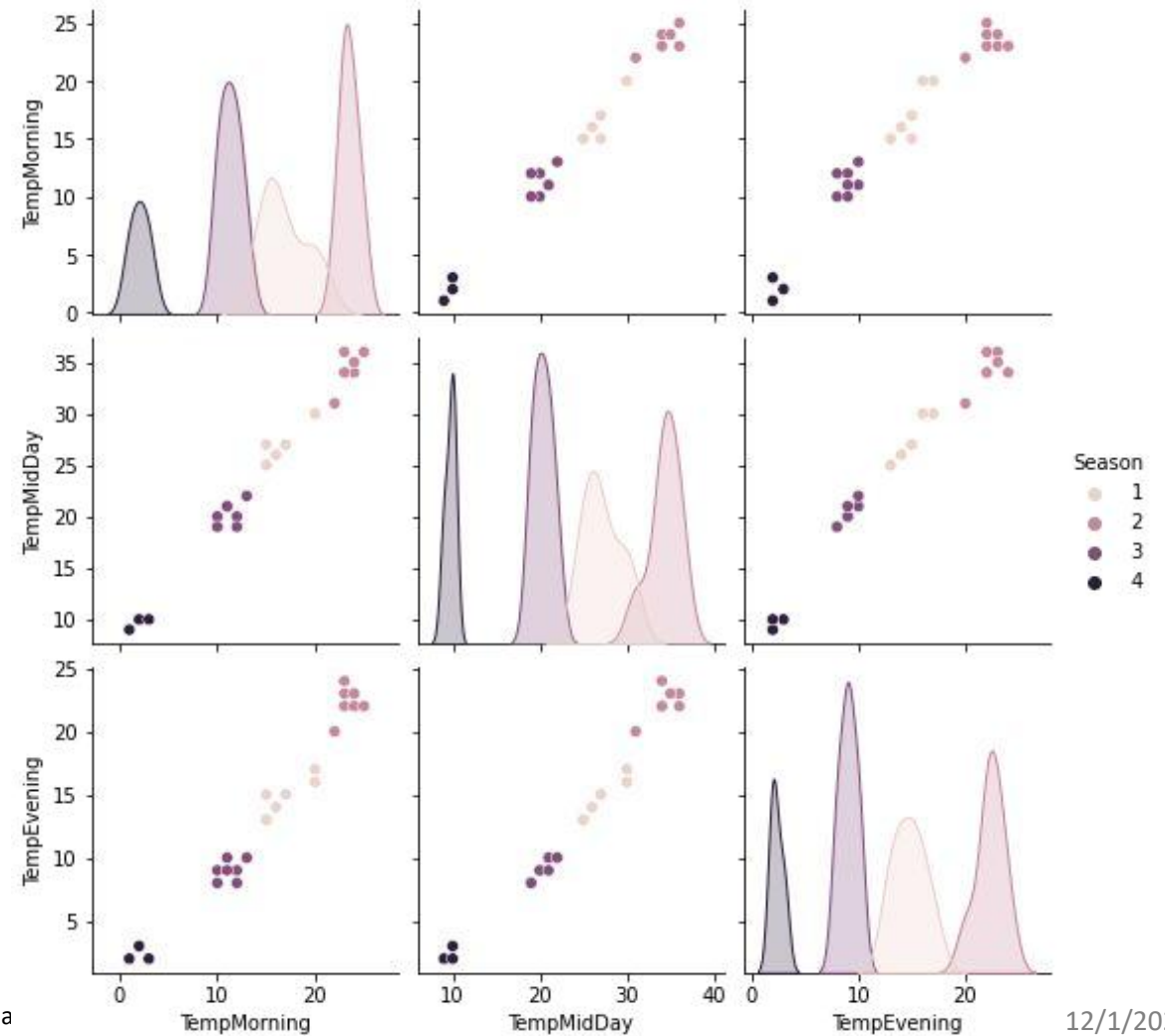
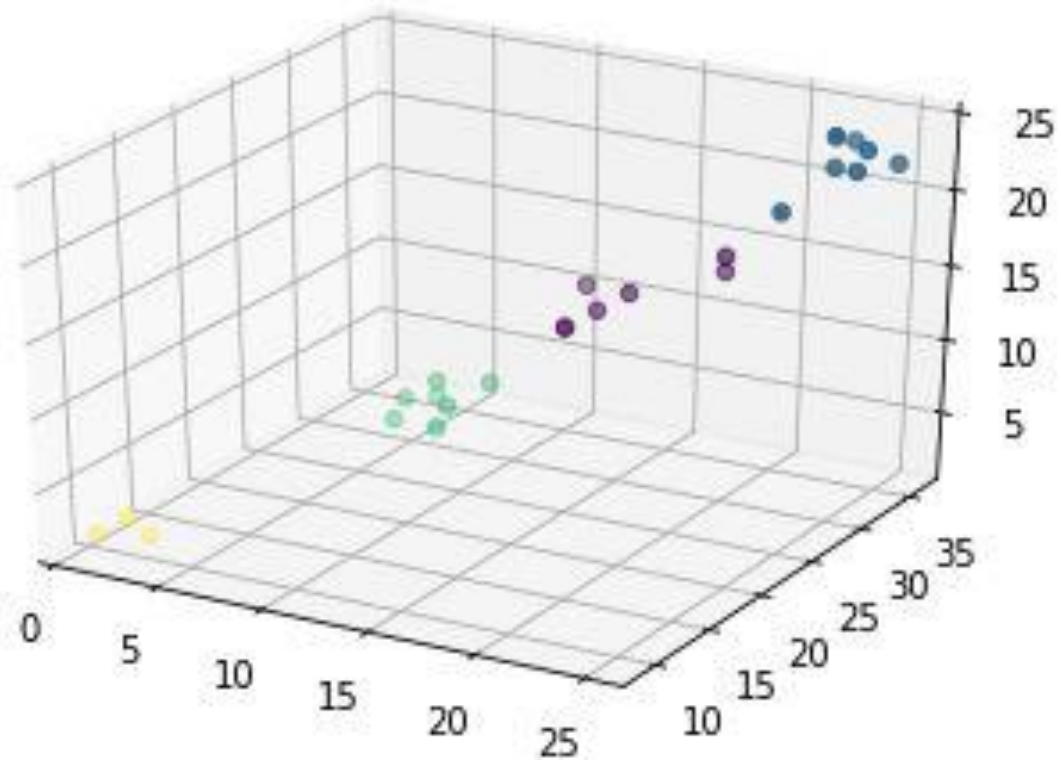
```

Demo



index ▲	TempMorning		TempMidDay		TempEvening		Season
0		15		25		13	1
1		16		26		14	1
2		15		27		15	1
3		17		27		15	1
4		20		30		17	1
5		20		30		16	1
6		15		25		13	1
7		22		31		20	2
8		23		34		22	2
9		24		34		22	2
10		23		36		23	2
11		25		36		22	2
12		23		34		24	2
13		24		35		23	2
14		10		20		9	3
15		11		21		10	3
16		10		19		8	3
17		12		20		9	3
18		13		22		10	3
19		12		19		8	3
20		11		21		9	3
21		2		10		3	4
22		1		9		2	4
23		3		10		2	4

Demo



Demo



```
def SMOTE(x, y, minority_class, k=5):
    """
    :param x: List[List[int]]
    :param y: List[List[int]]
    :param minority_class: int
    :param k: int
    :return: np.array, np.array
    """
    # Flatten target [[y]] -> [y]
    y = np.concatenate(y)

    # Find the minority class indices
    minority_indices = np.where(y == minority_class)[0]
    # Example: [21 22 23]

    # Find the nearest neighbors of the minority class points
    nn = NearestNeighbors(n_neighbors=k)
    nn.fit(x[minority_indices])
    _, indices = nn.kneighbors(x[minority_indices])
    # Example: [[0 2 1]
    #           [1 0 2]
    #           [2 0 1]]

    # Generate synthetic data points
    synthetic_data = []
    for i in range(len(minority_indices)):
        for j in indices[i]:
            synthetic_data.append(x[minority_indices[i]] + np.random.random() * (x[minority_indices[j]] - x[minority_indices[i]]))

    # Concatenate the synthetic data with the original data and labels
    X = np.concatenate((x, synthetic_data), axis=0)
    Y = np.concatenate((y, np.full(len(synthetic_data), minority_class)), axis=0)

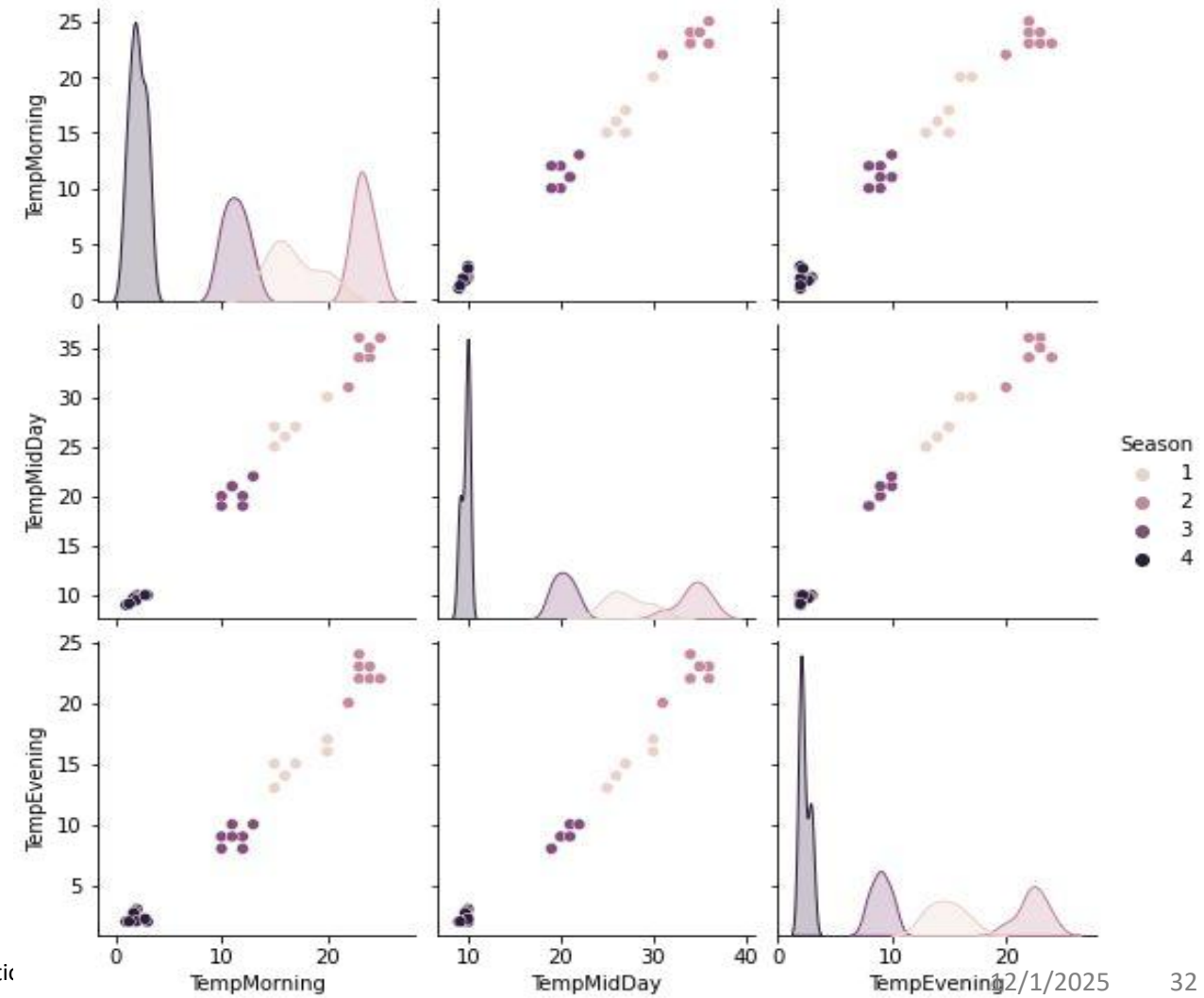
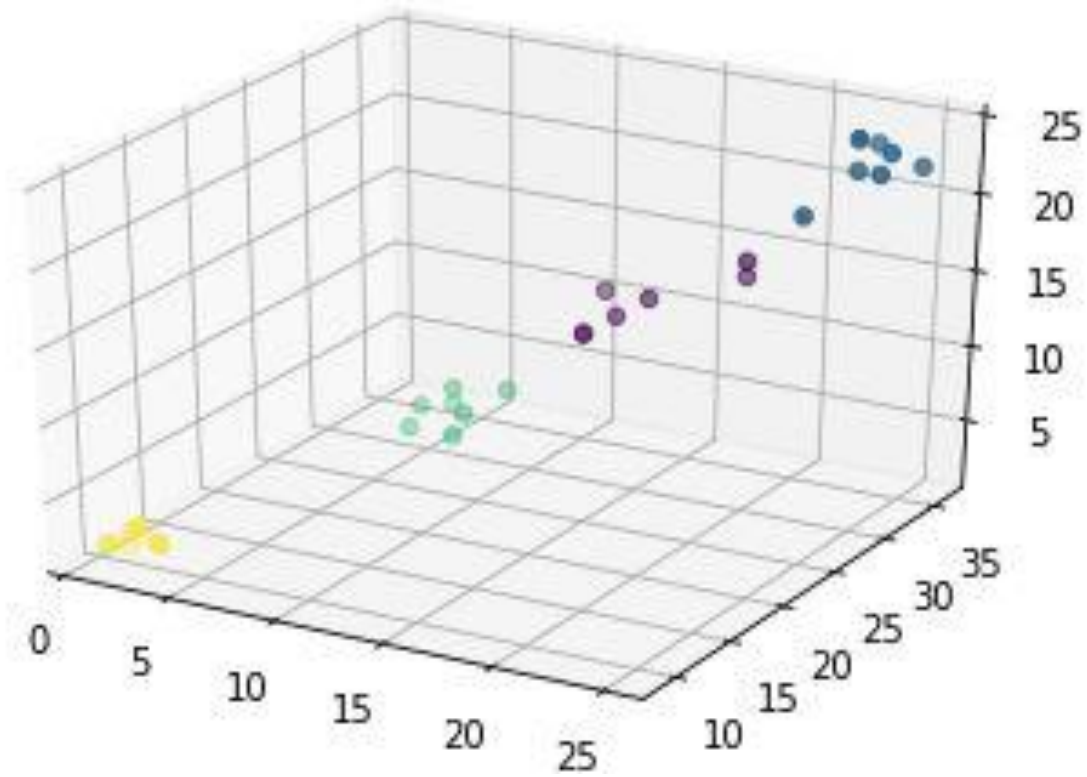
    return X, Y
```

Demo



index	TempMorning	TempMidDay	TempEvening	Season
0	15.0	25.0	13.0	1
1	16.0	26.0	14.0	1
2	15.0	27.0	15.0	1
3	17.0	27.0	15.0	1
4	20.0	30.0	17.0	1
5	20.0	30.0	16.0	1
6	15.0	25.0	13.0	1
7	22.0	31.0	20.0	2
8	23.0	34.0	22.0	2
9	24.0	34.0	22.0	2
10	23.0	36.0	23.0	2
11	25.0	36.0	22.0	2
12	23.0	34.0	24.0	2
13	24.0	35.0	23.0	2
14	10.0	20.0	9.0	3
15	11.0	21.0	10.0	3
16	10.0	19.0	8.0	3
17	12.0	20.0	9.0	3
18	13.0	22.0	10.0	3
19	12.0	19.0	8.0	3
20	11.0	21.0	9.0	3
21	2.0	10.0	3.0	4
22	1.0	9.0	2.0	4
23	3.0	10.0	2.0	4
24	2.0	10.0	3.0	4
25	2.824928248873837	10.0	2.175071751126163	4
26	1.8190481283097002	9.8190481283097	2.8190481283097	4
27	1.0	9.0	2.0	4
28	1.6895771225869012	9.6895771225869	2.6895771225869014	4
29	1.9144056826208546	9.457202841310428	2.0	4
30	3.0	10.0	2.0	4
31	2.783071430509759	10.0	2.216928569490241	4
32	1.269520734065127	9.134760367032564	2.0	4

Demo



Questions?

Dr. Sc. Mërgim H. HOTI

Faculty of Electrical and Computer Engineering,
University of Prishtina “Hasan PRISHTINA”,
Email: mergim.hoti@uni-pr.edu